

Применение признаков агрегации потоков на уровне источника (SourceStats) в детекторах сетевых аномалий

Заломный И.С.

Цель документа. Описать набор признаков, получаемых через агрегацию потоков на уровне источника (SourceStats), обосновать их применение в существующих и новых детекторах сетевых аномалий, и предложить пороговые значения для каждого решающего правила.

Отчёт является продолжением серии работ по адаптивным детекторам аномалий. В предыдущем отчёте коллеги-исследователя были рассмотрены статистические методы (скользящее среднее, перцентили, Z-score, MAD, EWMA) как адаптивный слой поверх существующих жёстких правил. Настоящий отчёт дополняет его, вводя ортогональный подход: вместо временной адаптации порогов — пространственная агрегация по источникам как новый слой данных, на котором строятся детекторы. Стат-примитивы из отчёта коллеги повторно используются в данной работе применительно к новым метрикам.

Анализ ведётся применительно к текущему состоянию проекта (репозиторий `bimka183/new_analizier`, ветка `main`): Go-бэкенд, парсер `rsar` на базе `gorocket`, разделение на потоки по 5-tuple и временные окна 10 секунд, набор детекторов в `src/detector`, хранилище SQLite, фронтенд на React, оффлайн-режим работы.

1 Обоснование подхода и выбора признаков

Почему агрегация по источнику. Текущие детекторы работают на двух уровнях данных: отдельный поток (FlowStats) и временное окно (WindowStats). Между ними остаётся пустота — поведение конкретного хоста как единого субъекта. Между тем, **большинство сетевых атак по своей природе определяются именно как паттерны поведения одного источника**: «один SrcIP стучится на 200 разных портов» (port scan), «один SrcIP касается 50 IP на порту 445» (распространение червя), «один SrcIP делает запросы каждые 60 секунд к одному внешнему серверу» (C2 beaconing). На уровне отдельного потока эти паттерны не видны — каждый поток выглядит безобидно. На уровне окна они видны как суммарная статистика, но без привязки к источнику невозможно отличить «один сканер» от «вся подсеть массово что-то делает». SourceStats — это агрегат всех потоков от одного SrcIP, который раскрывает поведение хоста как единое явление и делает атаку видимой там, где она по своей сути живёт.

Связь с отчётом по статистическим методам. Подход коллеги-исследователя — *временная* адаптация порогов: использовать историю окон для динамической оценки нормы. Подход данного отчёта — *пространственная* агрегация по источникам: использовать сравнение хостов между собой внутри одного `rsar` и сравнение хоста с его собственным профилем. Эти два подхода полностью

комплементарны: стат-примитивы из отчёта по статистическим методам (Z-score, p95, MAD, EWMA, относительный рост) одинаково применимы и к WindowStats, и к SourceStats. То есть SourceStats предоставляет метрики, а стат-примитивы коллеги — адаптивную оценку аномальности этих метрик. Вместе они образуют единый адаптивный слой: за «что считать аномалией» отвечает статистика, за «у кого аномалия» — агрегация по источникам.

Особенность офлайн-сценария. В оффлайне у нас есть весь pcap-файл целиком, что даёт два преимущества перед online-системами. Первое — мы можем построить **полный профиль источника по всему файлу** (per-pcap SourceStats), а не только по последнему окну. Это раскрывает медленные паттерны (slow scan на протяжении часа, медленное beaconing). Второе — **распределение метрик по всем источникам внутри pcap** становится естественным baseline для оценки выбросов: вместо «z-score хоста относительно его прошлого» мы получаем «z-score хоста относительно всех остальных хостов в этом же файле».

Какие признаки выбраны и почему.

- **Кардинальности назначений** (UniqueDstIPs, UniqueDstPorts, UniqueDstPairs) — базовые меры разброса поведения источника. Прямой сигнал для всех сканов и распространения червя.
- **Распределение connection states** (CntS0, CntREJ, CntSF и др.) — позволяет вычислить $\text{FailedRatio} = (S0 + REJ) / \text{Total}$. Это самый точный из имеющихся сигналов скана: легитимный клиент имеет $\text{FailedRatio} \approx 0.05-0.20$, сканер — $0.7-1.0$. Опирается на маппинг состояний из Zeek (см. отчёт по адаптации Zeek-подходов).
- **Концентрация на топ-К портах/IP** — отличает «целевую» активность (90% потоков на один порт = беспокойство о конкретном сервисе) от «разведывательной» (равномерное распределение по портам = enumeration). Концепция взята из работ по traffic profiling.
- **Энтропия Шеннона по портам и IP** — компактная численная мера концентрации/рассеянности поведения. Применяется в литературе по network anomaly detection.
- **Временные характеристики между потоками** — среднее, std и коэффициент вариации inter-flow interval. Низкий CV (<0.1) — признак регулярности (beaconing). Высокий CV — типичный человеческий клиент.
- **Балансы байтов** (BytesOut vs BytesIn) — для детекции data exfiltration и amplification participants.

Принцип комбинирования с детекторами. SourceStats не порождает алертов сам по себе. Каждый детектор использует поля SourceStats как дополнительные признаки в своём решающем правиле. Для каждого признака применяется двойная проверка: жёсткий порог (минимальный для предотвращения шума) и адаптивный (через стат-примитивы коллеги). Алерт срабатывает либо при пробитии жёсткого порога, либо при одновременном превышении адаптивного порога и наличии подтверждающих признаков (например, высокого FailedRatio).

2 Структура SourceStats

Файл: internal/packet/source_stats.go (новый)

Сущность строится на этапе постобработки потоков, после того как все FlowInfo собраны и каждый поток получил ConnState (см. отчёт по адаптации Zeek-подходов, §5.1).

```
1 type SourceStats struct {
2     SrcIP string
3
4     // --- Кардинальностиназначений ---
5     UniqueDstIPs int
6     UniqueDstPorts int
7     UniqueDstPairs int // уникальные пары (DstIP, DstPort)
8
9     // --- Объёмные характеристики ---
10    TotalFlows int
11    TotalPackets int64
12    TotalBytesOut int64 // отправленоисточником
13    TotalBytesIn int64 // полученоисточником
14
15    // --- Распределение connection states (Zeekмаппинг-) ---
16    CntS0, CntREJ, CntSF int
17    CntRST0, CntRSTR, CntOTH int
18    FailedRatio float64 // (CntS0 + CntREJ) / TotalFlows
19
20    // --- Профильназначений ---
21    TopDstPorts []PortCount // топ-5 dstпортов-
22    TopDstIPs []IPCount // топ-5 dst-IP
23    TopPortShare float64 // доля топ -1 портов TotalFlows
24    DstIPEntropy float64 // энтропияШеннонараспределения dst-IP
25    DstPortEntropy float64 // энтропияШеннонараспределения dst-port
26
27    // --- Временные характеристики ---
28    FirstSeen, LastSeen time.Time
29    Duration time.Duration
30    MeanInterFlowInterval float64 // секунды
31    StdInterFlowInterval float64
32    InterFlowCV float64 // коэффициентвариации = std/mean
33
34    // --- Категоризация ---
35    PortCategories map[string]int // wellknown / registered / dynamic / exploit
36    IsInternal bool // SrcIP принадлежит RFC1918 / HOME_NET
37 }
38
39 type PortCount struct { Port int; Count int }
40 type IPCount struct { IP string; Count int }
```

Для двух режимов работы нужны две коллекции:

```
1 // Per-pcap: один SourceStats на каждый SrcIP, агрегат по всему файлу
2 type PerPcapSources map[string]*SourceStats
3
4 // Per-window-per-source: SourceStats внутри каждого секундного 10- окна
5 // для детекции, опирающейся на динамику между окнами
6 type PerWindowSources map[WindowID]map[string]*SourceStats
```

Per-рсар агрегация раскрывает медленные паттерны (slow scan, beaconing на протяжении часа), per-window — быстрые (скачкообразный рост активности). Детекторы используют ту или другую в зависимости от типа атаки (см. §4).

3 Аналитические примитивы

Файл: `internal/sourcstats/primitives.go` (новый)

Стат-примитивы из отчёта по статистическим методам (Mean, StdDev, ZScore, Percentile, IQR, MAD, EWMA) **используются повторно** — SourceStats не дублирует их, а импортирует из `internal/stats/`. Дополнительно вводятся примитивы, специфичные для агрегации по источникам.

3.1 Энтропия Шеннона

$$H(X) = - \sum_{i=1}^n p_i \log_2 p_i, \quad p_i = \frac{\text{count}_i}{\sum_j \text{count}_j}$$

```
1 func ShannonEntropy(counts map[string]int) float64
```

Используется для оценки рассеянности распределения. Для k равных категорий энтропия равна $\log_2 k$. Для одной доминирующей категории (например, 95% потоков на один порт) — близка к 0.

Применение: разделять «сосредоточенное» поведение хоста (один порт = таргетная активность) и «разбросанное» (множество портов с равными долями = сканирование).

3.2 Топ-К с долей

```
1 func TopK(counts map[string]int, k int) []KeyCount
2 func TopKShare(counts map[string]int, k int) float64
```

Применение: вычисление TopPortShare и TopIPShare. Доминирующий порт со `share > 0.8` при большом числе уникальных IP — классическая сигнатура распространения червя.

3.3 Коэффициент вариации

$$CV = \frac{\sigma}{\mu}$$

```
1 func CoefficientOfVariation(values []float64) (float64, bool)
```

Применение: оценка регулярности inter-flow intervals. Beaconing-каналы дают $CV < 0.1$, человеческий трафик — обычно $CV > 0.5$.

3.4 Категоризация портов

```
1 func CategorizePort(port int) string
```

Категории: `wellknown` — 0-1023 (системные сервисы); `registered` — 1024-49151 (зарегистрированные приложения); `dynamic` — 49152-65535 (эфемерные/клиентские); `exploit` — особый список портов, исторически связанных с массовыми эксплоитами (445, 139, 1433, 3389, 23, 22 с условием `FailedRatio>0.5`, 5900, 6379, 27017). Список конфигурируем.

3.5 Принадлежность к внутренней сети

```
1 func IsInternalIP(ip string, homeNet []*net.IPNet) bool
```

Используется HOME_NET из конфигурации проекта (то же, что в WormDetector).
Применение: внешний источник, ведущий себя как сканер — высокий приоритет алерта; внутренний источник, ведущий себя как сканер на внутренние ресурсы — возможно lateral movement.

4 Применение в детекторах

Конвенция: каждое решающее правило формулируется как комбинация **жёсткого порога** (минимальный для предотвращения шума), **адаптивного порога** (через стат-примитивы коллеги-исследователя, например p95 распределения метрики среди всех источников рсар) и **подтверждающего признака** (как правило, FailedRatio).

4.1 Port scan

Файл: internal/detectors/portscan_detector.go

Текущая логика (предлагаемая в плане адаптации Zeek-подходов) — TRW-алгоритм по последовательности первых контактов. SourceStats обогащает её агрегированными признаками источника:

- **Жёсткий порог по уникальным портам.** UniqueDstPorts > 30 — стартовый порог. Обоснование: Nmap «quick scan» в дефолтной конфигурации пробует первые 100 наиболее популярных портов; даже усечённое «top-30» сканирование сразу попадает под эту границу. Легитимный клиент почти никогда не касается 30+ портов за окно.
- **Адаптивный порог.** UniqueDstPorts > max(30, p95) среди всех источников рсар. Учитывает профиль конкретного рсар: в рсар с NAT-шлюзом все клиенты могут иметь повышенный показатель, и фиксированный порог даст ложные срабатывания. Адаптивный порог — поверх жёсткого, не вместо.
- **Подтверждение через FailedRatio.** Сильный алерт срабатывает только если одновременно UniqueDstPorts > порог AND FailedRatio > 0.7. Это **главное** правило, отделяющее сканер от легитимного активного клиента. Обоснование: легитимные клиенты в среднем имеют FailedRatio < 0.2 (отдельные dropped connection-ы); сканер — близкое к 1.0 (большинство портов закрыты).
- **MAD-выброс среди источников.** Применение примитива из §2.4 отчёта коллеги: MADZScore(source.UniqueDstPorts, allSources.UniqueDstPorts) > 3.5. Робастно к ситуации, когда в рсар несколько сканеров одновременно (сами портят baseline для Z-score, но не для MAD).
- **Концентрация портов.** Если UniqueDstPorts > 30 и DstPortEntropy > $\log_2(\text{UniqueDstPorts}) - 1$ (близко к равномерному распределению по портам) — добавляется признак enumeration_scan. Если энтропия низкая при большом числе портов — это targeted probe-сет, не классический скан.

Пороги. MinUniqueDstPorts = 30; FailedRatioThreshold = 0.7; MADZScoreThreshold = 3.5 (берётся из отчёта коллеги).

4.2 Worm

Файл: internal/detectors/worm_detector.go

Текущая логика — список подозрительных портов ([445, 139, 1433, 6881, 25]) и проверка по internalNet. SourceStats радикально улучшает детектор: вместо «у потока подозрительный порт» получаем «источник демонстрирует паттерн распространения червя».

- **Жёсткий порог по уникальным IP.** UniqueDstIPs > 20. Совпадает с порогом addr_scan_threshold = 25 Zeek для address scan, слегка снижен потому что worm propagation часто медленнее, чем классический scan.
- **Концентрация на одном порту.** TopPortShare > 0.8 — более 80% потоков источника идут на один порт. Обоснование: червь распространяется через конкретную уязвимость, привязанную к конкретному порту. Сканер, в отличие от него, размазывает попытки по портам.
- **Топовый порт в exploit-категории.** TopDstPorts[0].Port ∈ {445, 139, 1433, 3389, 22, 23, ...}. Список ровно тот же, что в текущем WormDetector, но теперь применяется не к каждому отдельному потоку, а к доминирующему порту источника.
- **FailedRatio как индикатор сканирующего поведения.** Червь, сканируя сеть в поисках уязвимых машин, получает на большинстве IP отказы (REJ) или таймауты (S0). FailedRatio > 0.5 — характерный порог. Понижен относительно скана (0.7), так как у червя часть попыток заведомо успешна (заражённые машины отвечают нормально на тот же порт).
- **Относительный рост между окнами.** Если в предыдущем окне у источника UniqueDstIPs ≤ 5, в текущем — > 20, и при этом концентрация на одном exploit-порту: пометать как **начало пропагации**. Использует growth-ratio из §2.6 отчёта коллеги.
- **Внешний источник на внутреннюю сеть.** Если SrcIP внешний, а большинство DstIP — в HOME_NET, повышенный приоритет алерта (попытка проникновения, не lateral movement).
- **Внутренний на внутренний.** Если SrcIP внутренний и DstIP в основном внутренние, плюс exploit-порт: lateral movement, классическая картина WannaCry/NotPetya.

Решающее правило (сильный алерт):

```
UniqueDstIPs > 20
AND TopPortShare > 0.8
AND TopDstPorts[0].Port в exploit-списке
AND FailedRatio > 0.5
```

Если выполнены первые три условия, но FailedRatio < 0.5 — средний алерт (worm_candidate), требующий ручного просмотра.

Пороги. MinUniqueDstIPs = 20; MinTopPortShare = 0.8; WormFailedRatio = 0.5.

4.3 DDoS

Файл: `internal/detectors/ddos_detector.go`

Здесь нюанс: DDoS — это атака **многих источников на одну жертву**, и per-source агрегация на первый взгляд менее естественна, чем per-victim. Тем не менее SourceStats даёт два важных вклада: характеристика участников атаки и выявление amplification-актеров.

- **Идентификация бот-сети.** Среди всех источников, контактировавших с подозрительным dstIP за окно, кластеризуем по похожести SourceStats-профиля: близкие значения TotalPackets, TotalBytesOut, преобладающий ConnState, схожий MeanInterFlowInterval. Кластер размером >5 — высокая вероятность ботнета.
- **Amplification participants.** Источник, выступающий усилителем в атаке, имеет характерный профиль: UniqueDstIPs = 1 (одна жертва), порт источника в exploit-амп-списке (53/UDP, 123/UDP, 11211/UDP, 389/UDP, 1900/UDP), BytesOut $\gg$ BytesIn (огромный исходящий, маленький входящий). Если в рсар найдены 10+ таких источников, одновременно отправляющих к одному dstIP — сильный сигнал reflection-DDoS.
- **p99-фильтр «жертв»** (повторно используется из отчёта коллеги). Это уже описано у коллеги — оставляем как есть, но в качестве признаков для каждой жертвы используем агрегированные SourceStats всех её собеседников.
- **Per-source PPS-ранг.** Среди источников, говорящих с подозрительной жертвой, ранжируем по TotalPackets / Duration. Источники в топ-99-перцентиле — непропорциональные участники, кандидаты в бот-узлы.

Пороги. MinBotnetClusterSize = 5; AmplificationBytesRatio = 10.0 (BytesOut/BytesIn); MinAmplificationActors = 10.

4.4 C2 Beaconing (новый детектор, ставший возможным благодаря SourceStats)

Файл: `internal/detectors/beacon_detector.go` (новый)

Без SourceStats этот детектор не реализуется в принципе: одиночное соединение «раз в 60 секунд» по статистике потока неотличимо от любого другого. Нужна **серия соединений между одной парой (SrcIP, DstIP, DstPort) с регулярными интервалами**, что и даёт агрегация по источнику.

Алгоритм:

1. Для каждой тройки (SrcIP, DstIP, DstPort) собираем последовательность timestamps первых пакетов всех потоков.
2. Если число потоков $\geq$ MinBeaconRepeats (по умолчанию 5), считаем разности между последовательными timestamps.
3. Вычисляем $CV = \text{StdDev}(\text{intervals}) / \text{Mean}(\text{intervals})$.
4. Если $CV < 0.1$ и $\text{Mean}(\text{intervals}) > 10$ секунд (нижняя граница — отделить от нормального TCP keep-alive) — кандидат в beaconing.
5. Усиление сигнала: если DstIP внешний, размер пакетов мал (<1 KB), и DstPort не из списка типично-регулярных служб (NTP, DNS) — высокая вероятность C2.

Пороги. MinBeaconRepeats = 5; MaxBeaconCV = 0.1; MinBeaconInterval = 10s; MaxBeaconPayloadSize = 1024.

Обоснование порогов. $CV < 0.1$ — стандартная граница «высокой регулярности» в литературе по network anomaly; человеческий трафик практически никогда не даёт $CV < 0.3$. 5 повторов — минимум для оценки регулярности (3 точки — мало, 10 — слишком долгая задержка обнаружения). 10 секунд — нижний предел, исключающий keep-alive протоколы.

4.5 Overload

Файл: internal/detectors/overload_detector.go

SourceStats предоставляет **диагностический контекст** для существующего детектора перегрузки, не меняя его основное правило. К алерту перегрузки прикладывается:

- Топ-5 источников по TotalPackets / TotalBytes за окно перегрузки.
- Топ-5 dst-IP по входящим потокам в это окно.
- Признаки, отличающие легитимную перегрузку от атаки: при легитимной перегрузке топ-источники имеют диверсифицированный SourceStats-профиль (разные порты, разные dst-IP, низкий FailedRatio); при DDoS-обусловленной перегрузке — однотипный профиль и высокий FailedRatio.

Это не отдельное правило, а **обогащение контекста** для оператора.

5 Базовые правила и принципы применения

- **Жёсткий порог + подтверждение.** Любое срабатывание требует одновременно превышения жёсткого порога метрики И подтверждения через FailedRatio или другой характеризующий поведение признак. Голое превышение UniqueDstPorts > 30 без проверки FailedRatio — распространённая причина ложных срабатываний (NAT, P2P-клиенты, веб-сёрфинг).
- **Per-rcap baseline для адаптации.** Распределение SourceStats-метрик внутри одного rcap — естественный baseline для оценки выбросов. Применять стат-примитивы коллеги (MAD-z-score, p95) к этому распределению предпочтительнее, чем абсолютные пороги, потому что профиль rcap может сильно различаться (пограничный маршрутизатор vs офисная подсеть).
- **Минимум источников для адаптации.** Если в rcap менее MinSourcesForAdaptation = 10 уникальных активных источников — статистика распределения ненадёжна, работают только жёсткие пороги. Аналогично правилу MinSampleSize у коллеги.
- **Исключение текущего источника из baseline.** При оценке аномальности источника распределение всех остальных источников строится без него самого. Иначе сильный сканер вытягивает baseline вверх и сам себя нормализует. Прямой аналог правила «исключение текущего окна» у коллеги.
- **Робастные методы по умолчанию.** Для оценки выбросов среди источников MAD-z-score предпочтительнее обычного Z-score. В rcap часто присутствует более одного аномального источника, и они портят оценку среднего/ σ .

- **Сильный сигнал = жёсткий порог + адаптивный + подтверждение.**
Логика для каждого детектора: жёсткий алерт срабатывает если выполнены все три, и средний — если только два из трёх. Это позволяет сохранить высокую precision на сильных сигналах и не пропустить пограничные случаи.

6 Конфигурация

Параметр	Тип	Значение по умолчанию
MinSourcesForAdaptation	int	10
MinUniqueDstPortsScan	int	30
MinUniqueDstIPsScan	int	20
FailedRatioStrongScan	float	0.7
FailedRatioWormPropagation	float	0.5
MinTopPortShareWorm	float	0.8
ExploitPorts	[]int	445, 139, 1433, 3389, ...
AmplificationSrcPorts	[]int	53, 123, 161, 389, 1900, 11211
AmplificationBytesRatio	float	10.0
MinAmplificationActors	int	10
MinBotnetClusterSize	int	5
MinBeaconRepeats	int	5
MaxBeaconCV	float	0.10
MinBeaconInterval	duration	10s
MaxBeaconPayloadSize	int	1024
HomeNetCIDRs	[]string	из общей конфигурации

Все параметры выносятся в существующий конфиг (YAML/JSON), как и у коллеги; новых файлов конфигурации не вводить. Полный список ExploitPorts: {445, 139, 1433, 3389, 22, 23, 5900, 6379, 27017}.

7 Формат вывода

Каждое срабатывание детектора, использующего SourceStats, дополняется полем:

```

1 type SourceContext struct {
2     SrcIP          string
3
4     // Базовые метрики источника
5     UniqueDstIPs   int
6     UniqueDstPorts int
7     FailedRatio     float64
8     TopDstPort      int
9     TopDstPortShare float64
10    DstPortEntropy  float64
11
12    // Ранг среди источников   pcar
13    SourceRank             int    // 1 = самый активный
14    SourcePercentile       float64 // 0..100, по UniqueDstPorts
15    SourceMADZScore       float64 // робастный z-score
16
17    // Категоризация
18    IsInternal             bool
19    DominantPortCategory  string // "wellknown" / "exploit" / ...
20 }

```

Поле прикладывается ко всем алертам, где источник идентифицирован. Объединяется с `StatisticalContext` из отчёта коллеги (если применимы оба) в общий блок `AlertContext`. Это даёт оператору одновременно «по какой статистике сработало» (отчёт коллеги) и «у кого аномальное поведение» (данный отчёт).

8 Граничные случаи и ограничения

- **NAT-шлюзы.** Несколько хостов за одним публичным IP агрегируются в один `SourceStats`. Получаем профиль «суммарного клиента», который может выглядеть как сканер. Жёсткие пороги дадут ложные срабатывания. Митигация: пометить `SrcIP` как `behind_NAT` (если внешний и `UniqueSrcPorts` за окно > 50 — почти гарантированно NAT), снизить уверенность алерта. Для production-системы это решается через явное указание NAT-сетей в конфигурации.
- **Очень короткие или малонаселённые рсар.** Если в файле менее 10 уникальных источников — распределение метрик неинформативно, адаптивные пороги отключаются. Работают только жёсткие. Не баг, фича.
- **Асимметричный захват.** Если `capture` видит только одну сторону трафика, `BytesIn` и `BytesOut` искажены, `FailedRatio` считается неточно (некоторые ответы не видны). Проверка осуществимости: если $y > 50\%$ потоков нулевые `BytesIn` — помечаем рсар как `asymmetric_capture`, `FailedRatio`-зависимые правила понижают `confidence`.
- **Спуфинг SrcIP.** При спуфинге `SourceStats` теряет смысл (агрегируем по поддельным IP). Митигация: проверка через `ConnState`-распределение: при спуфинге большинство — `S0` (ответы не доходят до источника). Если у источника $\text{CntS0} > 0.95 \times \text{TotalFlows}$ и `UniqueDstIPs` очень мала — кандидат на `spoofed source`, отдельный сценарий обработки.
- **Очень активные легитимные хосты.** Веб-серверы, CDN-узлы, прокси, P2P-клиенты могут иметь огромные `UniqueDstIPs/UniqueDstPorts`, не являясь атакующими. Основная защита — `FailedRatio` как подтверждение: легитимные активные хосты всё равно имеют низкий `FailedRatio`. Дополнительно — белые списки `well-known cloud-IP`, если это критично для конкретного развертывания.
- **Производительность.** Per-рсар агрегация — $O(N_{flows})$ по памяти и времени. Per-window — $O(N_{flows} \times N_{windows})$ в худшем случае, но на практике каждый поток попадает в одно окно. Расчёт энтропий и top-K — линейный по числу уникальных назначений источника, не превышает нескольких сотен. На стандартных рсар (десятки тысяч потоков) накладные расходы — менее секунды.

9 Тестирование

К существующим фикстурам (см. отчёты по TLS-метаданным и по статистическим методам) добавить:

1. `nmap_horizontal_scan.pсар` — один внешний источник сканирует 50 IP по портам 22, 80, 443. Должен сработать `port scan` по `UniqueDstPorts > 30 AND FailedRatio > 0.7`.

2. `wannacry_propagation.pcap` — внутренний хост стучится на 50 внутренних IP по порту 445 с `FailedRatio` ≈ 0.6 . Должен сработать worm-детектор по комбинации правил §4.2.
3. `dns_amplification.pcap` — 15 источников с порта 53/UDP отправляют по 200 пакетов размером 4KB на одну жертву, `BytesIn` от жертвы — единицы байт. Должен сработать `amplification`.
4. `beacon_60s.pcap` — внутренний хост отправляет соединение раз в 60 ± 2 секунды на внешний IP:8443, всего 12 повторов. Должен сработать `beacon`-детектор.
5. `legit_high_activity.pcap` — внутренний веб-сервер обрабатывает 100 клиентов с `RPS=10`, `FailedRatio` ≈ 0.05 . Не должно быть ложных срабатываний ни у `port scan`, ни у `worm`.
6. `nat_gateway.pcap` — за одним публичным IP сидят 30 хостов, генерирующих разнообразный веб-трафик. Должен сработать признак `behind_NAT` без алерта об атаке.
7. `spoofed_source.pcap` — источник со 100% S0-потоками. Должен быть помечен как кандидат `spoofed`, обычные алерты подавлены.

Дополнительно — юнит-тесты на стат-примитивы §3 (Shannon entropy, CV, категоризация портов) с эталонными входами.

10 Дальнейшие улучшения (вне MVP)

- **Графовый анализ.** `SourceStats` отвечает на «кто что делал», графовый анализ — на «кто с кем связан». Построение графа коммуникаций (`SrcIP`, `DstIP`) с весами по объёму и поиск аномальных подграфов (`cluster-структура ботнета`, `центр-периферия` для C2) — следующий уровень детекции.
- **Кросс-рсар профили.** Если в системе доступен корпус `рсар`-файлов, для известных хостов (внутренних серверов) можно строить долгосрочные эталонные профили `SourceStats` и алертить на отклонение текущего `рсар` от исторического профиля хоста. Требуется базы профилей; не для MVP.
- **JA3/JA4-обогащение.** При добавлении DPI к `SourceStats` логично добавить распределение JA3/JA4 `fingerprint`-ов источника. Один хост с одним JA3 — норма; один хост со множеством разных JA3 в коротком окне — подозрительно. Требуется расширения парсера; не для MVP.
- **Кластеризация источников через ML.** Текущее предложение — простая кластеризация участников ботнета по близости численных метрик. Полноценная ML-кластеризация (DBSCAN по нормализованному вектору `SourceStats`-метрик) даст более точное выделение групп. Требуется подбора фич и калибровки; следующая итерация.
- **Поведенческие классы хостов.** Авто-классификация `SrcIP` в классы (`workstation`, `server`, `scanner`, `beacon`, `p2p`, `web-crawler`) на основе сочетания признаков `SourceStats`. Полезно для дальнейшей фильтрации алертов по контексту хоста.

Источники

- Karagiannis T., Papagiannaki K., Faloutsos M. "BLINC: Multilevel Traffic Classification in the Dark", ACM SIGCOMM 2005.
- Lakhina A., Crovella M., Diot C. "Mining Anomalies Using Traffic Feature Distributions", ACM SIGCOMM 2005, pp. 217-228.
- Gu G., Zhang J., Lee W. "BotSniffer: Detecting Botnet Command and Control Channels in Network Traffic", NDSS 2008 — обоснование подхода «низкий CV inter-flow intervals = beaconing».
- Akoglu L., Tong H., Koutra D. "Graph based anomaly detection and description: a survey", Data Mining and Knowledge Discovery, vol. 29, no. 3, 2015, pp. 626-688.
- Jung J., Paxson V., Berger A.W., Balakrishnan H. "Fast Portscan Detection Using Sequential Hypothesis Testing", IEEE Symposium on Security and Privacy, 2004 — алгоритм TRW, используемый в §4.1.
- Paxson V. "Bro: A System for Detecting Network Intruders in Real-Time", Computer Networks, vol. 31, no. 23-24, 1999, pp. 2435-2463 — принцип per-host профилирования.